

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический университет»
Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 — «Информатика и вычислительная
техника»

Отчёт по практической работе на тему
**«Разработка автоматизированного рабочего места специалиста по учёту
бартерных обменов в общежитии»**

Выполнили студенты гр. ИВТ-25-16

Рахмангулов Динар Дамирович

Ражапов Марсель Рафаэлевич

Проверил:

(оценка) (подпись)

(дата)

г. Пермь, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ	4
1.1 Основные требования	4
1.2 Проектирование	5
1.2.1 Классы	5
1.2.1.1 Класс BarterModel	5
1.2.1.2 Класс TradeManager	6
1.2.1.3 Класс MainWindow	6
1.2.1.4 Класс ChainFinder	7
1.2.1.5 Класс Storage	7
1.2.2 Модули визуализации	7
1.2.3 Диалоговые окна	8
2 ИСПОЛЬЗУЕМЫЕ ИНСТРУМЕНТЫ	9
2.1 Используемые библиотеки	9
2.1.1 Фреймворк Qt	9
2.1.2 Модули Qt JSON	9
2.1.3 OpenGL	9
2.1.4 Стандартная библиотека C++	9
2.2 Другие инструменты	10
ЗАКЛЮЧЕНИЕ	11
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	13

ВВЕДЕНИЕ

Актуальность работы:

В условиях проживания в общежитии студенты часто сталкиваются с необходимостью обмена вещами, учебными материалами, бытовыми предметами и другими ресурсами. Ведение таких обменов вручную неудобно: сложно отслеживать владельцев, состояние предметов, историю завершённых сделок и возможные варианты взаимовыгодного обмена. Разработанное автоматизированное рабочее место позволяет добавлять предметы для обмена, хранить сведения о владельцах, учитывать желаемые варианты обмена, оформлять сделки и находить цепочки обменов между несколькими участниками.

Цель работы:

Целью проекта является создание настольного приложения для учёта бартерных обменов в общежитии с функциями хранения списка предметов, оформления обменов, ведения истории сделок, визуализации распределения предметов по категориям и автоматического поиска цепочек обмена.

Задачи:

- 1) Определить, какие библиотеки будут использоваться в проекте.
- 2) Продумать структуру проекта.
- 3) Создать функции добавления и удаления предметов для обмена.
- 4) Создать функции оформления бартерной сделки между двумя участниками.
- 5) Реализовать хранение истории обменов и восстановление данных после перезапуска программы.
- 6) Написать алгоритм поиска цепочек обмена на основе предпочтений участников.
- 7) Реализовать окно программы с помощью фреймворка Qt.
- 8) Реализовать графическую визуализацию статистики и цепочек обмена.
- 9) Протестировать работу проекта.

10) Составить отчёт о проделанной работе.

1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ

1.1 Основные требования

Перед непосредственной разработкой были определены основные требования к работе программы:

- Пользователь добавляет предмет для обмена через отдельное диалоговое окно.
- Программа хранит следующую информацию о предмете:
 - название;
 - описание;
 - имя владельца;
 - комната владельца;
 - категория предмета;
 - текущий статус;
 - пожелание владельца о том, что он хочет получить взамен;
 - дата и время добавления предмета.
- Пользователь может удалить предмет из списка.
- Для оформления обмена пользователь выбирает два доступных предмета.
- После подтверждения обмена оба предмета получают статус «Обменян».
- Программа сохраняет историю всех совершённых обменов.
- Программа показывает подробную информацию о выбранном предмете.
- Программа отображает статистику количества предметов по категориям.
- Программа выполняет поиск цепочек обмена между несколькими участниками.
- Все данные сохраняются в JSON-файл barterdata.json и загружаются при следующем запуске.

1.2 Проектирование

При проектировании была выбрана модульная структура проекта. Основная логика разделена на классы, отвечающие за данные предметов, таблицу отображения, оформление обменов, поиск цепочек, сохранение информации и графическую визуализацию.

1.2.1 Классы

В программе используются следующие пользовательские классы и структуры:

- Item;
- TradeRecord;
- BarterModel;
- TradeManager;
- MainWindow;
- ItemDialog;
- ChainDialog;
- ChainFinder;
- Storage;
- GLWidget;
- GraphWidget.

1.2.1.1 Класс BarterModel

Класс BarterModel наследуется от QAbstractTableModel и отвечает за хранение списка предметов и отображение этих данных в таблице. В модели реализованы методы rowCount, columnCount, data и headerData, которые позволяют таблице получать количество строк, количество столбцов, значения ячеек и заголовки.

Класс предоставляет функции addItem, removeItem, updateItemStatus, getItem и loadItems. С их помощью выполняется добавление предметов, удаление строк, изменение статуса предмета после обмена и загрузка данных

при запуске программы. Также реализован метод `categoryCounts`, который подсчитывает количество предметов по категориям для дальнейшей визуализации.

1.2.1.2 Класс TradeManager

Класс `TradeManager` управляет историей обменов. Основная функция `makeTrade` принимает идентификаторы двух предметов, проверяет их наличие и доступность, после чего изменяет статус обоих предметов на «Обменян» и создаёт запись `TradeRecord`.

В записи обмена сохраняются идентификатор сделки, идентификаторы предметов, имена участников и дата совершения обмена. После успешного оформления сделки класс отправляет сигнал `tradeCompleted`, который используется главным окном для обновления списка истории.

1.2.1.3 Класс MainWindow

Класс `MainWindow` реализует главное окно программы и связывает все части приложения. В нём создаются таблица предметов, поле подробной информации, список истории обменов, кнопки управления, строка состояния и OpenGL-виджет визуализации.

Основные функции класса `MainWindow`:

- `buildUI` — создание интерфейса приложения;
- `onAddItem` — добавление нового предмета через диалоговое окно;
- `onRemoveItem` — удаление выбранного предмета;
- `onMakeTrade` — оформление обмена между двумя выбранными предметами;
- `onFindChains` — поиск и оформление цепочек обмена;
- `onItemSelectedChanged` — отображение подробной информации о выбранном предмете;
- `updateStats` — обновление статистики;
- `saveData` — сохранение предметов и истории в файл;
- `closeEvent` — сохранение данных перед закрытием приложения.

1.2.1.4 Класс ChainFinder

Класс ChainFinder реализует алгоритм поиска бартерных цепочек. Для поиска используется ориентированный граф: узлом является доступный предмет, а ребро означает, что владелец одного предмета хочет получить предмет другого участника.

Совместимость предметов определяется функцией `wantsItem`. Если поле «Хочу взамен» не заполнено, совпадение проверяется по категории. Если поле заполнено, программа сравнивает ключевые слова из пожелания с названием, описанием и категорией предлагаемого предмета.

Метод `buildGraph` строит граф смежности, а метод `dfs` выполняет поиск циклов длиной от 2 до заданного максимального значения. Найденные циклы преобразуются в структуру `BarterChain`, где каждый шаг описывает, кто отдаёт предмет, какой предмет передаётся и кто его получает.

1.2.1.5 Класс Storage

Класс Storage отвечает за сохранение и загрузку данных. Для хранения используется файл `barterdata.json`, расположенный в каталоге приложения. В файл записываются два массива: `items` и `history`.

Для преобразования данных используются функции `itemToJson`, `itemFromJson`, `tradeToJson` и `tradeFromJson`. Предметы и записи обмена сохраняются в виде JSON-объектов, что делает структуру файла наглядной и удобной для проверки.

1.2.2 Модули визуализации

Для визуализации в проекте используются классы `GLWidget` и `GraphWidget`.

`GLWidget` отображает диаграмму количества предметов по категориям. Для каждой категории используется собственный цвет. Виджет строит столбцы, сетку и подписи, а также использует таймер для плавной анимации появления диаграммы.

GraphWidget отображает найденные цепочки обмена в виде графа. Узлы соответствуют участникам и предметам, а стрелки показывают направление передачи предмета. При выборе цепочки программа подсвечивает соответствующие рёбра и узлы.

1.2.3 Диалоговые окна

ItemDialog используется для ввода информации о новом предмете. В диалоге расположены поля названия, описания, имени владельца, комнаты, категории и желаемого предмета взамен. Перед добавлением выполняется проверка обязательных полей.

ChainDialog используется для просмотра найденных цепочек обмена. Диалог содержит список цепочек, текстовое описание выбранной цепочки и вкладку с графической визуализацией. После подтверждения выбранная цепочка оформляется как последовательность обменов.

2 ИСПОЛЬЗУЕМЫЕ ИНСТРУМЕНТЫ

2.1 Используемые библиотеки

2.1.1 Фреймворк Qt

Qt — кроссплатформенный фреймворк для разработки GUI-приложений на C++. В проекте были использованы следующие модули:

- QtWidgets для создания главного окна, таблиц, списков, кнопок и диалоговых окон;
- QtCore для работы с датой, временем, сигналами, слотами и контейнерами;
- QtGui для работы с цветами, шрифтами и элементами отображения;
- Qt OpenGL для реализации пользовательских виджетов визуализации.

Qt выбран благодаря удобству разработки интерфейса, поддержке объектной модели, механизму сигналов и слотов, а также возможности создавать настольные приложения с понятной структурой.

2.1.2 Модули Qt JSON

Для сохранения и загрузки данных используются классы QJsonObject, QJsonArray и QJsonDocument. С их помощью программа преобразует предметы и записи обменов в JSON-формат и записывает их в файл barterdata.json.

2.1.3 OpenGL

OpenGL используется для построения графической визуализации. В проекте реализованы два виджета на основе QOpenGLWidget: диаграмма количества предметов по категориям и граф цепочек обмена.

2.1.4 Стандартная библиотека C++

Стандартная библиотека C++ применяется для базовых алгоритмов и операций. В проекте используется сортировка найденных цепочек, обработка

данных и вспомогательная логика, необходимая для работы алгоритма поиска.

2.2 Другие инструменты

Также при разработке проекта использовались следующие инструменты:

- среда разработки Qt Creator;
- язык программирования C++;
- файл проекта qmake Barter_Exchange_2.pro;
- Qt Designer для работы с UI-файлами;
- JSON-файл barterdata.json для хранения данных приложения.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы разработано автоматизированное рабочее место для учёта бартерных обменов в общежитии. Цель работы достигнута полностью.

Для достижения цели были решены следующие задачи:

- выбран и использован фреймворк Qt для разработки графического интерфейса;
- разработана структура данных для хранения предметов и записей обменов;
- реализована таблица предметов с отображением владельца, комнаты, категории, статуса и желаемого предмета взамен;
- созданы функции добавления и удаления предметов;
- реализовано оформление обмена между двумя участниками;
- создана история завершённых обменов;
- разработан алгоритм поиска цепочек обмена на основе ориентированного графа и DFS;
- реализовано сохранение и восстановление данных в JSON-файле;
- создана OpenGL-визуализация статистики по категориям и графа обменных цепочек.

В результате создано полнофункциональное приложение, позволяющее удобно вести учёт предметов, фиксировать бартерные сделки и находить возможные цепочки обмена между участниками.

Программа протестирована на основных пользовательских сценариях. Проверены добавление предмета, удаление предмета, оформление обмена, поиск цепочек, отображение статистики, сохранение данных и повторная загрузка после перезапуска.

Все основные сценарии работают корректно. Пользовательский интерфейс интуитивно понятен и не требует специального обучения.

Перспективы дальнейшего развития:

- добавление редактирования уже созданных предметов;
- расширение фильтрации и поиска по категориям;
- экспорт истории обменов;
- улучшение алгоритма подбора цепочек обмена;
- добавление общего каталога предметов для работы нескольких пользователей.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Страуструп Б. Язык программирования C++. — Москва: Бином, 2020.
2. Бланшет Ж., Саммерфилд М. C++ GUI Programming with Qt 4. — Prentice Hall, 2008.
3. Qt Documentation [Электронный ресурс]. — Режим доступа: <https://doc.qt.io/> (дата обращения: 15.05.2026).
4. OpenGL Documentation [Электронный ресурс]. — Режим доступа: <https://www.opengl.org/documentation/> (дата обращения: 15.05.2026).
5. ГОСТ 7.32–2017. Отчёт о научно-исследовательской работе. Структура и правила оформления.